Universidad Nacional Autónoma de México

Computer Engineering

Compilers

Parser and SDT

Team 7

Students:

- 321189536
- 321184191
- 117002933
- 321134567
- 321335630

Group:

5

Semester:

2026-II

Mexico, CDMX. April 28 2026

Contents

1 Introduction **2**

1.1 Problem Statement 2

1.2 Motivation 2

1.3 Objectives 2

2 Theoretical Framework **3**

3 Body **6**

3.1 Parser 6

3.2 FIRST and FOLLOW Sets 9

3.3 LL(1) Parsing Table 11

3.4 Proof that the grammar is LL(1) 12

3.5 Parser implementation 15

3.6 Abstract Syntax Tree (AST) 16

3.7 Semantic Analyzer 18

3.8 Why Top-Down and not Bottom-Up 21

4 Results **21**

5 Conclusions **27**

1. Introduction

1.1 Problem Statement

The compilation process can be divided into a sequence of analyses that progressively transform source code from raw text into a structured representation.
In the previous course project, a lexical analyzer for the C language was developed, whose purpose was to correctly identify and classify the basic lexical elements (tokens) present in the source code, such as identifiers, keywords, and operators.
The next phase consists of verifying that these tokens form a well-structured program and that the program complies with the semantic rules of the language. Both aspects are addressed in this project through the implementation of a predictive recursive-descent parser, together with a Syntax-Directed Translation (SDT) scheme for semantic validation, culminating in the generation and visualization of an Abstract Syntax Tree (AST).

1.2 Motivation

Syntax analysis is a fundamental stage in compiler design, as it verifies that the sequence of tokens produced by the lexical analyzer conforms to the grammatical structure of the language. Implementing a parser allows a deeper understanding of how programming languages are structured and how hierarchical relationships between constructs are defined.
Furthermore, incorporating a Syntax-Directed Translation (SDT) scheme enables the integration of semantic validation into the parsing process, allowing the detection of errors such as undeclared variables, type mismatches, and incorrect use of language constructs. This stage also facilitates the construction of structured representations such as Abstract Syntax Trees (ASTs), which are essential for subsequent phases of compilation.

1.3 Objectives

- Define an unambiguous, left-factored, left-recursion-free context-free grammar for a subset of C covering declarations, assignments, control flow, function definitions and calls, and arithmetic, comparison and logical expressions with operator precedence.
- Prove formally that the grammar is LL(1) by computing its FIRST and FOLLOW sets and showing that no production conflicts arise at any decision point.
- Implement a Top-Down predictive recursive-descent parser that consumes the token stream produced by the lexer and produces a leftmost derivation along with an

attributed AST.

- Integrate semantic actions (an SDT) inside the parser to detect: redeclaration of identifiers, use of undeclared identifiers, type incompatibility in declarations, assignments, expressions and `return` statements, `void` functions returning a value, and non-void functions missing a return value.
- Provide a graphical user interface that exposes the three official outcomes required by the assignment: `Parsing Success! / SDT Verified!`, `Parsing error... / SDT error...` and `Parsing Success! / SDT error...`.

2. Theoretical Framework

Lexical Analyzer

A lexical analyzer is a program that converts the input text into tokens and classifies these tokens into predefined categories [1]. The lexical analyzer plays a crucial role in the analysis phase, since it provides the tokens to the parser to facilitate syntax analysis. In this project, the lexical analyzer reads a string or file containing C code and identifies the tokens that compose the program. [2]

Token

A token is a sequence of characters grouped into a single entity. Each token represents a set of character sequences that convey a specific meaning within the programming language. Tokens are the basic units that allow the compiler to understand the structure of the source code [2].

Token Classification

In this project, the lexical analyzer classifies tokens into the following categories:

- Keywords
- Identifiers
- Constants
- Literals
- Operators
- Punctuation
- Unknown tokens [3]

Keywords correspond to reserved words of the C language. Identifiers represent the names assigned to variables, functions, or other program elements. Constants represent numeric or character values that do not change during program execution. Literals represent sequences of characters enclosed in quotation marks. Operators represent arithmetic, logical, relational, or assignment operations. Punctuation symbols are characters used to structure the syntax of the program. Unknown tokens represent sequences of characters that cannot be classified into any valid token category [4].

Parser

A parser, or syntactic analyzer, is the second phase of a compiler. It receives the token stream produced by the lexer and decides whether the sequence of tokens forms a sentence in the language defined by a given context-free grammar. The output of a successful parse is a derivation or, equivalently, a parse tree that exposes the hierarchical structure of the program. Parsers are classified into *Top-Down* parsers, which build the tree from the start symbol toward the input, and *Bottom-Up* parsers, which build it from the input toward the start symbol [5].

Top-Down Parsing (LL(1))

A Top-Down parser builds the parse tree from the root down to the leaves, reading the input from left to right and producing a *leftmost derivation*. An LL(1) parser looks one token ahead and decides which production to apply using a predictive parsing table built from the FIRST and FOLLOW sets [6].

FIRST and FOLLOW sets

For a grammar symbol X, $\text{FIRST}(X)$ is the set of terminals that can begin some string derived from X; if X can derive the empty string, ε is also included. For a non-terminal A, $\text{FOLLOW}(A)$ is the set of terminals that may appear immediately to the right of A in some sentential form; the end-of-input marker \$ belongs to $\text{FOLLOW}(S)$ for the start symbol S.

A grammar is **LL(1)** if, for every pair of productions $A \to \alpha$ and $A \to \beta$:

- $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$, and
- whenever $\varepsilon \in \text{FIRST}(\alpha)$, also $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$ [7].

Syntax-Directed Translation (SDT)

A Syntax-Directed Translation is a context-free grammar enriched with *semantic rules* associated with its productions. Each grammar symbol carries one or more *attributes*; the rules describe how those attributes are computed during parsing. Attributes can be *synthesized* (computed from the children of a node) or *inherited* (computed from the parent or siblings). Typical uses of an SDT include type checking, symbol-table construction, expression evaluation and the generation of intermediate code.

In our project the SDT is woven directly into the recursive-descent parser: each parsing procedure performs its semantic action as it consumes tokens, populating a stack of scoped symbol tables and verifying type compatibility on the fly [8].

3. Body

3.1 Parser

The development of the Parser was carried out in several stages. First, a context-free grammar based on a subset of the C language was defined. This grammar was then transformed through left factoring and elimination of left recursion to make it compatible with LL(1) predictive parsing. Subsequently, the FIRST and FOLLOW sets were computed in order to verify the absence of ambiguities and ensure that the grammar was suitable for the construction of a recursive-descent parser.
Based on this theoretical foundation, the syntactic analyzer was implemented using a Top-Down approach, where each production of the grammar is represented by a recursive procedure. During this process, syntax-directed translation (SDT) rules were also integrated to perform semantic validation, such as correct variable usage, type compatibility, and scope management.

3.1.1 Context Free Grammar (CFG)

The syntactic structure of the language is defined using a Context-Free Grammar (CFG), derived from a simplified subset of the C programming language. This grammar was specifically designed to be compatible with LL(1) predictive parsing, ensuring the absence of ambiguities during the syntactic analysis process.
The grammar allows the declaration of global and local variables, as well as their optional initialization. It supports basic data types such as `int`, `float`, `double`, `char`, and `void` (only used for functions). Additionally, it allows the definition of functions without parameters and the construction of statement blocks.
The language also supports conditional control structures such as `if` and `if-else`, as well as return and assignment statements. Furthermore, it includes support for arithmetic, relational, and logical expressions, with precedence levels defined through the grammar hierarchy (E, TERM, FACTOR, UNARY, etc.).
The grammar is composed of four fundamental components: terminals, non-terminals, production rules, and the start symbol.
Terminals represent the basic lexical units of the language, provided by the lexical analyzer. They include reserved keywords such as `int`, `float`, `double`, `char`, `if`, `else`, and `return`, as well as identifiers, constants, literals, arithmetic operators, relational operators, logical operators, and punctuation symbols. Terminals represent the leaves of the parse tree and cannot be further decomposed.
Non-terminals define the syntactic structure of the language. They represent higher-level constructs such as programs, global and local declarations, statements, expressions, and control structures. These abstract symbols are expanded during

syntactic analysis through the application of production rules.

Production rules describe how non-terminals can be replaced by combinations of terminals and other non-terminals. These rules define the valid structure of programs in the language, including variable declarations, function definitions without parameters, conditional statements, assignments, return statements, and expressions with different levels of precedence and associativity.

To enable predictive parsing, the grammar was transformed by eliminating left recursion and applying left factoring, ensuring that each non-terminal can be parsed deterministically using a single lookahead token.

Start symbol is defined as `PROGRAM`, which represents the entry point of the language. From this symbol, all valid programs are derived through successive applications of production rules.

Language restrictions (derived from the grammar):

- Functions do not accept parameters, as the corresponding production only allows `()` without argument lists.
- Loop structures such as `for` and `while` are not supported, since they are not defined in the grammar.
- Arrays, pointers, and complex data structures are not supported.
- Standard input/output operations are not supported.
- Function calls are limited to the syntax without arguments, according to the `STMT_ID_REST` production.
- Any construct not explicitly defined in the grammar is considered invalid during syntactic analysis.

These restrictions are intentional to maintain the grammar in LL(1) form, eliminating ambiguities and enabling the construction of an efficient recursive-descent parser. This also facilitates the integration with semantic analysis and the generation of the Abstract Syntax Tree (AST).

Listing 1: C-Pure context-free grammar

```
PROGRAM          → GLOBAL PROGRAM | epsilon

GLOBAL           → TYPE id GLOBAL_REST
TYPE             → int | float | double | char | void

GLOBAL_REST      → ( ) { STMT_LIST }
                 | OPT_ASSIGN ;
```

```
OPT_ASSIGN       → = E | epsilon

STMT_LIST        → STMT STMT_LIST | epsilon

STMT             → IF_STMT
                 | return OPT_E ;
                 | TYPE id OPT_ASSIGN ;
                 | id STMT_ID_REST

STMT_ID_REST     → = E ;
                 | ( ) ;

OPT_E            → E | epsilon

IF_STMT          → if ( E ) { STMT_LIST } ELSE_PART

ELSE_PART        → else { STMT_LIST } | epsilon


E                → LOGIC_OR

LOGIC_OR         → LOGIC_AND LOGIC_OR_PRIME
LOGIC_OR_PRIME   → || LOGIC_AND LOGIC_OR_PRIME | epsilon

LOGIC_AND        → EQUALITY LOGIC_AND_PRIME
LOGIC_AND_PRIME  → && EQUALITY LOGIC_AND_PRIME | epsilon

EQUALITY         → COMPARISON EQUALITY_PRIME
EQUALITY_PRIME   → == COMPARISON EQUALITY_PRIME
                 | != COMPARISON EQUALITY_PRIME
                 | epsilon

COMPARISON       → TERM COMPARISON_PRIME
COMPARISON_PRIME → > TERM COMPARISON_PRIME
                 | < TERM COMPARISON_PRIME
                 | >= TERM COMPARISON_PRIME
                 | <= TERM COMPARISON_PRIME
                 | epsilon

TERM             → FACTOR TERM_PRIME
TERM_PRIME       → + FACTOR TERM_PRIME
```

```
                  | - FACTOR TERM_PRIME
                  | epsilon

FACTOR            → UNARY FACTOR_PRIME
FACTOR_PRIME      → * UNARY FACTOR_PRIME
                  | / UNARY FACTOR_PRIME
                  | % UNARY FACTOR_PRIME
                  | epsilon

UNARY             → ! UNARY
                  | - UNARY
                  | PRIMARY

PRIMARY           → id
                  | constant
                  | literal
                  | ( E )
```

3.2 FIRST and FOLLOW Sets

To construct the LL(1) predictive syntactic analyzer, the FIRST and FOLLOW sets were calculated for each non-terminal in the grammar. These sets are fundamental for ensuring deterministic production selection during the parsing process, thereby eliminating ambiguities and potential conflicts when choosing derivation rules.

The **FIRST** set of a non-terminal comprises all terminal symbols that can appear at the beginning of strings derived from that non-terminal, including the empty string (ϵ) for nullable productions.

Conversely, the **FOLLOW** set contains the set of terminal symbols that can appear immediately to the right of a non-terminal in any valid derivation. The end-of-input marker (`$`) is, by definition, included in the FOLLOW set of the start symbol, `PROGRAM`.

FIRST Sets

Non-Terminal	**FIRST Set**
PROGRAM	{ char, double, float, int, void, ϵ }
GLOBAL, TYPE	{ char, double, float, int, void }
GLOBAL_REST	{ (, ;, = }
OPT_ASSIGN	{ =, ϵ }
STMT_LIST	{ char, double, float, id, if, int, return, void, ϵ }
STMT	{ char, double, float, id, if, int, return, void }
STMT_ID_REST	{ (, = }
OPT_E	{ !, (, -, constant, id, literal, ϵ }
IF_STMT	{ if }
ELSE_PART	{ else, ϵ }
E, LOGIC_OR, LOGIC_AND, EQUALITY	{ !, (, -, constant, id, literal }
COMPARISON, TERM, FACTOR, UNARY	{ !, (, -, constant, id, literal }
PRIMARY	{ (, constant, id, literal }
LOGIC_OR_PRIME	{ ||, ϵ }
LOGIC_AND_PRIME	{ &&, ϵ }
EQUALITY_PRIME	{ !=, ==, ϵ }
COMPARISON_PRIME	{ <, <=, >, >=, ϵ }
TERM_PRIME	{ +, -, ϵ }
FACTOR_PRIME	{ %, *, /, ϵ }

Table 1: FIRST sets for the C-Pure grammar.

FOLLOW Sets

Non-Terminal	**FOLLOW Set**
PROGRAM	{ $ }
GLOBAL, GLOBAL_REST	{ $, char, double, float, int, void }
TYPE	{ id }
OPT_ASSIGN, OPT_E	{ ; }
STMT_LIST	{ } }
STMT, STMT_ID_REST, IF_STMT, ELSE_PART	{ char, double, float, id, if, int, return, void,
E, LOGIC_OR, LOGIC_OR_PRIME	{), ; }
LOGIC_AND, LOGIC_AND_PRIME, ...	{ &&,), ;, || }
COMPARISON, COMPARISON_PRIME, ...	{ !=, &&,), ;, ==, <, <=, >, >=, || }
FACTOR, FACTOR_PRIME	{ !=, &&,), +, -, ;, ==, <, <=, >, >=, || }
UNARY, PRIMARY	{ !=, %, &&,), *, +, -, /, ;, ==, <, <=, >, >=, |

Table 2: FOLLOW sets for the C-Pure grammar.

The calculated FIRST and FOLLOW sets confirm that the grammar is suitable for LL(1) predictive parsing, as there are no selection conflicts between alternative productions. Furthermore, the systematic handling of ϵ within the FIRST sets allows for the correct parsing of optional language constructs. These sets serve as the mathematical foundation for the parsing table, ensuring the deterministic behavior of the recursive-descent parser.

3.3 LL(1) Parsing Table

The LL(1) parsing table is a two-dimensional matrix $M[A, a]$, where A represents a non-terminal symbol and a represents a terminal symbol or the end-of-input marker ($). This table serves as the lookup mechanism for the predictive parser, determining which production rule to apply based on the current non-terminal at the top of the stack and the lookahead token provided by the lexical analyzer.

Construction Methodology

The construction of the parsing table was performed systematically using the previously computed FIRST and FOLLOW sets. For each production rule of the form $A \to \alpha$, the following procedure was applied:

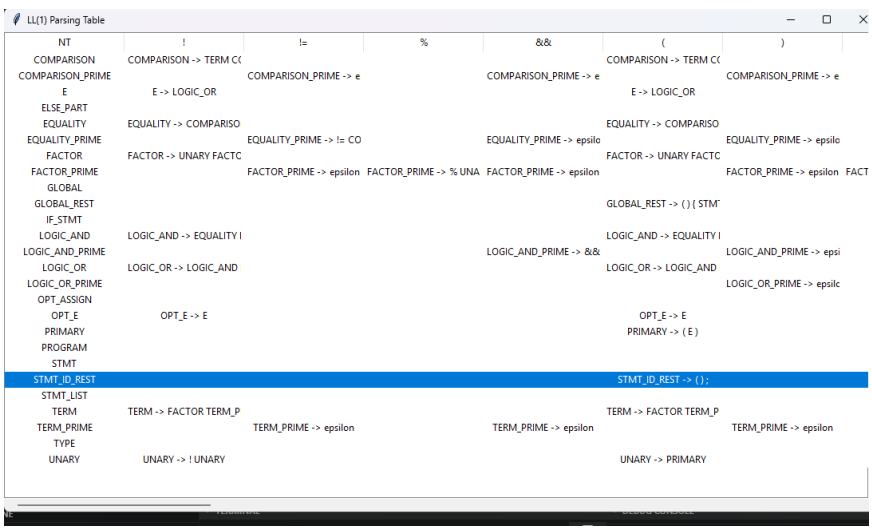
- **Direct Terminals:** For every terminal $a \in FIRST(\alpha)$, the production $A \to \alpha$ is inserted into the entry $M[A, a]$.

- **Handling Epsilon (ϵ):** If $\epsilon \in FIRST(\alpha)$, then for every terminal $b \in FOLLOW(A)$, the production $A \to \alpha$ is added to $M[A, b]$.
- **End of Input:** If $\epsilon \in FIRST(\alpha)$ and $ (end-of-input marker) is in $FOLLOW(A)$, then the production $A \to \alpha$ is also added to $M[A, \$]$.

Conflict Resolution and Determinism

A key objective during the construction process was to ensure that the grammar remains LL(1)-compliant. This is verified by the fact that no cell in the table contains more than one production rule. The absence of shift-reduce and reduce-reduce conflicts confirms that:

- For any non-terminal A, the FIRST sets of its alternative productions are pairwise disjoint.
- If a non-terminal can derive ϵ, then its FIRST set does not overlap with its FOLLOW set.



LL(1) Parsing Table

NT	!	!=	%	&&	(	)
COMPARISON	COMPARISON -> TERM C(				COMPARISON -> TERM C(	
COMPARISON_PRIME		COMPARISON_PRIME -> e		COMPARISON_PRIME -> e		COMPARISON_PRIME -> e
E	E -> LOGIC_OR				E -> LOGIC_OR	
ELSE_PART						
EQUALITY	EQUALITY -> COMPARISO				EQUALITY -> COMPARISO	
EQUALITY_PRIME		EQUALITY_PRIME -> != CO		EQUALITY_PRIME -> epsilo		EQUALITY_PRIME -> epsilo
FACTOR	FACTOR -> UNARY FACTC				FACTOR -> UNARY FACTC	
FACTOR_PRIME		FACTOR_PRIME -> epsilon	FACTOR_PRIME -> % UNA	FACTOR_PRIME -> epsilon		FACTOR_PRIME -> epsilon FACT
GLOBAL						
GLOBAL_REST					GLOBAL_REST -> ( ) { STMT	
IF_STMT						
LOGIC_AND	LOGIC_AND -> EQUALITY I				LOGIC_AND -> EQUALITY I	
LOGIC_AND_PRIME				LOGIC_AND_PRIME -> &&		LOGIC_AND_PRIME -> epsi
LOGIC_OR	LOGIC_OR -> LOGIC_AND				LOGIC_OR -> LOGIC_AND	
LOGIC_OR_PRIME						LOGIC_OR_PRIME -> epsilc
OPT_ASSIGN						
OPT_E	OPT_E -> E				OPT_E -> E	
PRIMARY					PRIMARY -> (E)	
PROGRAM						
STMT						
STMT_ID_REST					STMT_ID_REST -> ( ) ;	
STMT_LIST						
TERM	TERM -> FACTOR TERM_P				TERM -> FACTOR TERM_P	
TERM_PRIME		TERM_PRIME -> epsilon		TERM_PRIME -> epsilon		TERM_PRIME -> epsilon
TYPE						
UNARY	UNARY -> ! UNARY				UNARY -> PRIMARY	

3.4 Proof that the grammar is LL(1)

A grammar is LL(1) if for every pair of alternatives $A \to \alpha \mid \beta$ the following conditions hold:

(i) $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$

(ii) If $\varepsilon \in \text{FIRST}(\alpha)$ then $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$

We verify the grammar by analyzing each decision point.

Program structure —

- PROGRAM → GLOBAL PROGRAM | ε:

  FIRST(GLOBAL PROGRAM) = FIRST(GLOBAL) = {int, float, double, char, void}

  FOLLOW(PROGRAM) = {$}

  Therefore:

  $$\text{FIRST}(GLOBAL) \cap \text{FOLLOW}(PROGRAM) = \emptyset$$

  Condition (ii) holds.

—

Global declarations —

- GLOBAL → TYPE id GLOBAL_REST

  FIRST(GLOBAL) = FIRST(TYPE) = {int, float, double, char, void}

  TYPE alternatives are disjoint:

  $$\text{FIRST}(int) \cap \text{FIRST}(float) \cap \cdots = \emptyset$$

- GLOBAL_REST:

  – ( ){$STMT_LIST$} (function)

  – OPT_ASSIGN ; (variable)

  FIRST sets:

  $$\{(\} \cap \{=, ;, \varepsilon\} = \emptyset$$

  Therefore no conflict exists.

Statements —

- STMT has alternatives:

  – IF_STMT → starts with **if**

  – return → starts with **return**

  – TYPE id → starts with TYPE

  – id STMT_ID_REST → starts with id

  FIRST sets are disjoint:

  $$\{if\} \cap \{return\} \cap \{int, float, double, char, void\} \cap \{id\} = \emptyset$$

- STMT_ID_REST:
  - = E ;
  - ( )

  FIRST sets:

$$\{=\} \cap \{(\} = \emptyset$$

Conditional statements —

- IF_STMT → if (E) { STMT_LIST } ELSE_PART

  ELSE_PART:
  - else { STMT_LIST }
  - ε

  FOLLOW(ELSE_PART) = {if, return, TYPE, id, }}

$$\{else\} \cap FOLLOW(ELSE_PART) = \emptyset$$

  Condition (ii) satisfied.

Statement list —

- STMT_LIST → STMT STMT_LIST |

  FOLLOW(STMT_LIST) = { , else, }

  FIRST(STMT) = {if, return, TYPE, id}

$$\text{FIRST}(STMT) \cap \text{FOLLOW}(STMT_LIST) = \emptyset$$

Expressions (LL(1) hierarchy) —Each level follows:

$$X \to YX' \quad , \quad X' \to op\ Y\ X' \mid \varepsilon$$

- LOGIC_OR_PRIME: {||}
- LOGIC_AND_PRIME: {}
- EQUALITY_PRIME: {==, !=}
- COMPARISON_PRIME: {>, <, >=, <=}
- TERM_PRIME: {+, -}
- FACTOR_PRIME: {*, /, %}

All FOLLOW sets exclude their operators, so:

$$\text{FIRST}(op) \cap \text{FOLLOW}(X') = \emptyset$$

Thus LL(1) conditions hold.

Unary and primary —

- UNARY → !UNARY | -UNARY | PRIMARY

  FIRST sets:

  $$\{!, -\} \cap FIRST(PRIMARY) = \emptyset$$

- PRIMARY:
  - id
  - constant
  - literal
  - (E)

  All alternatives start with distinct terminals.

Conclusion

All production rules satisfy LL(1) conditions: no FIRST/FIRST or FIRST/FOLLOW conflicts exist.
Therefore, the grammar is LL(1) and can be parsed using a predictive recursive-descent parser without backtracking.

3.5 Parser implementation

The parser is implemented in Python as a recursive descent parser based on an LL(1)-style grammar. It is structured as a class named `Parser`, where each method represents a non-terminal symbol of the language grammar.
The parsing process is driven by a stream of tokens generated by the lexical analyzer. The parser uses helper functions such as `peek()` to inspect the current token, `advance()` to consume tokens, and `match()` or `consume()` to validate expected symbols. If a token does not match the expected grammar rule, a syntactic error is raised with the corresponding line number.

The parsing process begins in the method `parse_program`, which represents the start symbol of the grammar. From there, the parser iteratively processes global declarations using the method `parse_global`, which distinguishes between variable declarations and function declarations based on the lookahead token.

Global variable declarations are handled by `parse_global_var`, while function declarations are processed by `parse_function`, which also introduces a new scope for local symbols.
The method `parse_statement` handles all possible statements inside functions or blocks, including conditional statements (`if`), return statements, local variable declarations, assignments, and function calls. The selection of the appropriate production rule is based on the current token.

The expression grammar is implemented using a set of recursive methods that enforce operator precedence and associativity. These include `parse_expression`, `parse_logic_or`, `parse_logic_and`, `parse_equality`, `parse_comparison`, `parse_term`, `parse_factor`, `parse_unary`, and `parse_primary`.
Each method corresponds to a level in the expression hierarchy, ensuring correct parsing of arithmetic, logical, and relational operations.

In summary, the parser performs syntactic analysis by validating the structure of the input program according to the defined grammar and constructing the corresponding hierarchical representation through recursive descent parsing.

3.6 Abstract Syntax Tree (AST)

The Abstract Syntax Tree is the intermediate representation produced by the parser during recursive descent. While the leftmost derivation records *which* productions were applied, the AST captures the *hierarchical structure* of the program in a form that the semantic analyzer can traverse efficiently. Every production in the grammar that consumes meaningful syntactic information emits an `ASTNode`; auxiliary primed non-terminals (`TERM_PRIME`, `FACTOR_PRIME`, `LOGIC_OR_PRIME`, etc.) do not produce nodes of their own — they collapse into the parent expression node, which keeps the tree shallow and readable.

3.6.1 Node structure

Each node is an instance of the `ASTNode` class defined in `src/main/Ast/ASTNode.py`.
A node carries four attributes:

- `node_type` (*string*): the syntactic category of the node, for example `PROGRAM`, `FUNCTION`, `ASSIGN`, `IF`, `BIN_OP`, `UNARY`, `ID`, `CONST` or `LITERAL`. This attribute drives both visualization and downstream semantic analysis.
- `children` (*list of ASTNode*): the ordered list of child nodes. The order matches left-to-right appearance in the source code, which preserves the information needed for code generation in later phases.

- `value` (*string, optional*): the lexeme or operator associated with the node when applicable, for example `"x"` for an identifier, `"+"` for a binary operator, or `"int main"` for a function declaration.
- `inferred_type` (*string, optional*): the data type computed by the SDT during parsing (`int`, `float`, `double`, `char` or `void`). This attribute is the bridge between the syntactic and the semantic phases: it is set by the parser as a synthesized attribute and consulted by every type-compatibility check.

The class also overrides `__repr__` to produce a hierarchical, indented textual rendering of the tree, useful for debugging, console output, and the *Syntactic* view of the GUI.

3.6.2 Construction during parsing

AST construction is interleaved with parsing: every `parse_*` method in the `Parser` class returns an `ASTNode` that summarises the subtree rooted at that non-terminal. For example, `parse_local_decl` returns a `LOCAL_VAR` node whose child is the optional initializer expression; `parse_if` returns an `IF` node whose three children are the condition expression, the then-block, and the optional else-block; and the binary expression methods (`parse_term`, `parse_factor`, etc.) emit `BIN_OP` nodes whose `value` is the operator and whose `inferred_type` is the result of `_infer` applied to the operand subtrees.

This design means the AST is fully populated by the time parsing finishes: there is no separate AST-building pass.

3.6.3 Visualization

The module `src/main/Ast/ast_visualizer.py` exposes a `render_ast(ast, filename, file_format)` function that converts any AST into a Graphviz `Digraph` and renders it as an image (PNG by default). The visualizer walks the tree once, assigns a unique identifier to each node, builds a label that combines `node_type`, `value` and `inferred_type`, and applies a colour code that helps the reader identify the role of each node at a glance, as shown in Table 3.

Node category	Examples	Fill colour
Constants and literals	`CONST`, `LITERAL`	light green
Identifiers	`ID`	light blue
Variable declarations	`LOCAL_VAR`, `GLOBAL_VAR`	light orange
Operators	`BIN_OP`, `UNARY`	light red
Functions	`FUNCTION`	light purple
Other (`IF`, `RETURN`, etc.)	—	white

Table 3: Colour mapping used by the AST visualizer.

The visualizer is decoupled from the parser: it only depends on the public attributes of `ASTNode` (`node_type`, `value`, `inferred_type`, `children`). This keeps the parser free of any rendering dependency, and allows the AST to be reused by future phases (intermediate code generation, optimization) without modification.

3.6.4 Why an AST and not just a derivation

A leftmost derivation is sufficient to prove that a program belongs to the language defined by the grammar, but it is not convenient for semantic analysis. The derivation is a flat sequence of production applications; reasoning about "the type of the right-hand side of this assignment" from a derivation alone requires reconstructing nested context. The AST collapses this nesting into an explicit tree where every parent–child relationship is a real semantic dependency.

As a result, the SDT can be expressed as a local operation on each node: the `inferred_type` of a parent is a pure function of the `inferred_type`s of its children. This locality is precisely what makes the recursive-descent implementation clean: each `parse_*` method is responsible for one node, and the type rules compose naturally as the recursion unwinds.

AST as the bridge between syntax and semantics

The AST is what allows our compiler to be both **Top-Down** and **Syntax-Directed**. The parser produces it, the SDT consumes it, and future phases of the compiler will traverse it again to generate intermediate code. It is the central data structure that ties every phase of the compiler together.

3.7 Semantic Analyzer

The development of the Semantic Analyzer was carried out after the construction of the Abstract Syntax Tree (AST) generated by the parser. Its main objective is to

perform semantic validation of the program by traversing the AST, ensuring that the code complies with the rules of the language beyond its syntactic structure. In this implementation, the analyzer verifies the correct usage of variables, functions, classes, and control structures, as well as type consistency in expressions, assignments, and return statements.
The semantic analyzer uses a symbol table to store information about declared identifiers, including their type, scope, and initialization state. Based on this information, it performs checks such as detection of undeclared identifiers, redefinition of variables or functions, use of uninitialized variables, and type compatibility errors. It also supports type inference in expressions to validate complex operations. All detected errors are recorded with their corresponding line numbers, facilitating debugging and producing as output either a structured list of semantic errors or a confirmation that the program is semantically correct.

3.7.1 Symbol Table

The Symbol Table is a fundamental component of the semantic analysis phase and is responsible for managing all declared identifiers in the program, including variables, functions, and classes. It is implemented as a scoped data structure that allows the correct resolution of identifiers depending on their visibility and lifetime within the program.
In this implementation, the symbol table is designed as a stack of dictionaries, where each dictionary represents a different scope. When a new block is entered (such as a function or a control structure like `if`), a new scope is pushed onto the stack using the `enter_scope()` operation. When the block is exited, the scope is removed using `exit_scope()`. This mechanism ensures proper handling of nested scopes and prevents identifiers from being accessed outside their valid context.
Each entry in the symbol table stores essential information about an identifier, including its name, type, and additional attributes such as whether it has been initialized or whether it represents a function. This allows the semantic analyzer to perform detailed validation during parsing and expression evaluation.

The main responsibilities of the symbol table in this system are the following:

- **Declaration management:** It stores identifiers when they are declared and prevents redeclaration within the same scope.
- **Scope resolution:** It resolves identifiers by searching from the innermost scope to the outermost scope, ensuring correct visibility rules.
- **Initialization tracking:** It keeps track of whether variables have been initialized before use, allowing detection of uninitialized variable usage.

- **Function handling:** It distinguishes between variables and functions, storing function metadata such as return type and enabling validation of function calls.

During semantic analysis, the parser interacts directly with the symbol table to insert new identifiers and validate their correct usage. For example, before inserting a new variable, the table checks for duplicates in the current scope. Similarly, when an identifier is used in an expression or assignment, the table is queried to verify its existence and retrieve its type information.

3.7.2 SDT specification

The Syntax-Directed Translation (SDT) is directly integrated into the recursive-descent parser. Instead of using an external translation phase, semantic actions are executed during parsing, allowing syntax and semantics to be processed simultaneously in a single pass.
The SDT is supported by a symbol table that manages identifiers across different scopes using an internal stack mechanism (implemented through the `enter_scope()` and `exit_scope()` operations). Each function declaration introduces a new scope, while global declarations remain in the outermost scope. In addition, the parser maintains the attribute `current_function_type`, which is used to validate return statements and ensure consistency between function signatures and returned expressions.

The semantic information is carried using synthesized attributes stored directly in AST nodes. Each expression-related node contains an `inferred_type` attribute, which is computed during parsing using helper rules such as `_infer` for arithmetic operations. Terminal values such as identifiers, constants, and literals are classified using lexical information (`Identifiers`, `Constants`, `Literals`) and converted into typed AST nodes. These attributes are then propagated upward through the parse tree to validate type compatibility and semantic correctness.

The main semantic actions embedded in the parser are summarized below:

- Variable declaration: When a variable is declared, it is inserted into the symbol table. If the identifier already exists in the current scope, a redeclaration error is reported. If an initializer is present, its type is checked against the declared type.
- Assignment statements: The left-hand identifier is resolved through the symbol table. If it is not found, an undeclared variable error is generated. Otherwise, the type of the assigned expression is compared with the variable type, and mismatches are reported.
- Function declarations: Each function is inserted into the symbol table with its return type and marked as a function entry. A new scope is created for its body.

The parser ensures that non-void functions return a value and that return types match the declared function type.

- Function calls: The symbol table is queried to verify that the identifier corresponds to a declared function. If not, an error is raised. The call inherits the function's return type as its inferred type.
- Expressions: Binary and unary operations compute their resulting type using type inference rules. Arithmetic operations promote types when necessary (e.g., `int` with `float` results in `float`), while logical and comparison operations produce `int` as boolean representation.

3.8 Why Top-Down and not Bottom-Up

Three considerations led us to choose Top-Down recursive descent over a Bottom-Up shift-reduce parser:

1. The grammar is LL(1) after explicit left-factoring (notably the `<STMT_TAIL>` and `<PRIMARY_TAIL>` factors introduced to disambiguate the common `id` prefix in statements and primary expressions). Once factored, recursive descent maps the grammar 1-to-1 to code without any extra machinery: no item sets, no GOTO/ACTION tables.
2. The semantic actions of our SDT — which need access to scoped symbol tables and propagate inferred types — are most naturally expressed as procedure parameters (inherited attributes such as `current_function_type`) and return values (synthesized attributes such as the `inferred_type` on every `ASTNode`). In LR parsing, semantic actions can only fire at reductions and require an explicit value stack, complicating the exposition without expressive gain.
3. The leftmost derivation produced by the parser matches the order in which a programmer reads the source code, which makes the trace shown by our GUI straightforward to follow during the project presentation and easier to debug during development.

4. Results

The project exhibits four distinct outcomes demonstrating the compiler's robustness across different stages of analysis: lexical, syntactic, and semantic.

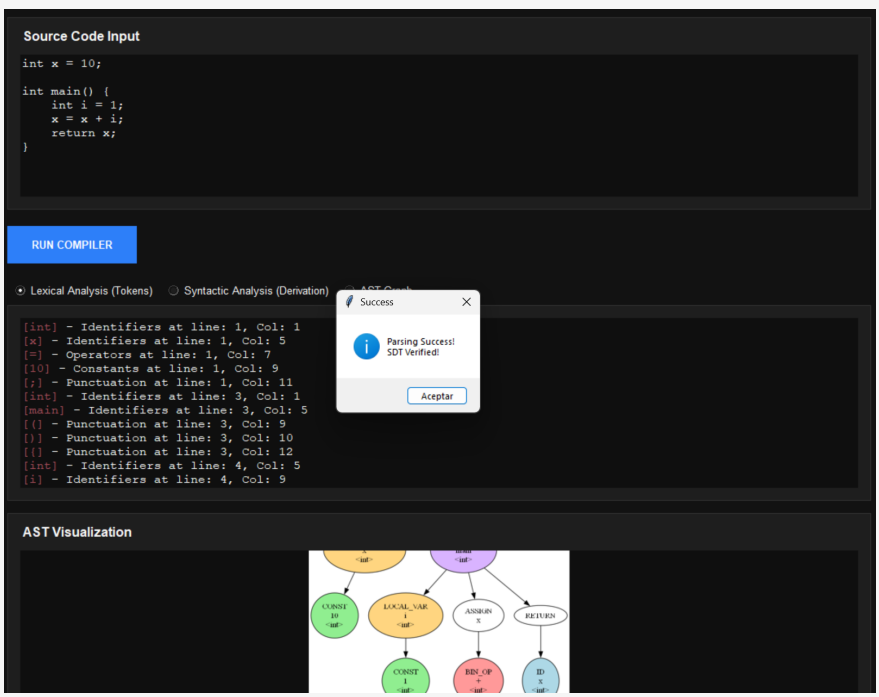
Outcome 1 — Successful compilation

For a syntactically and semantically correct input that includes a global variable, a local variable, an arithmetic operation, and a `return` statement:

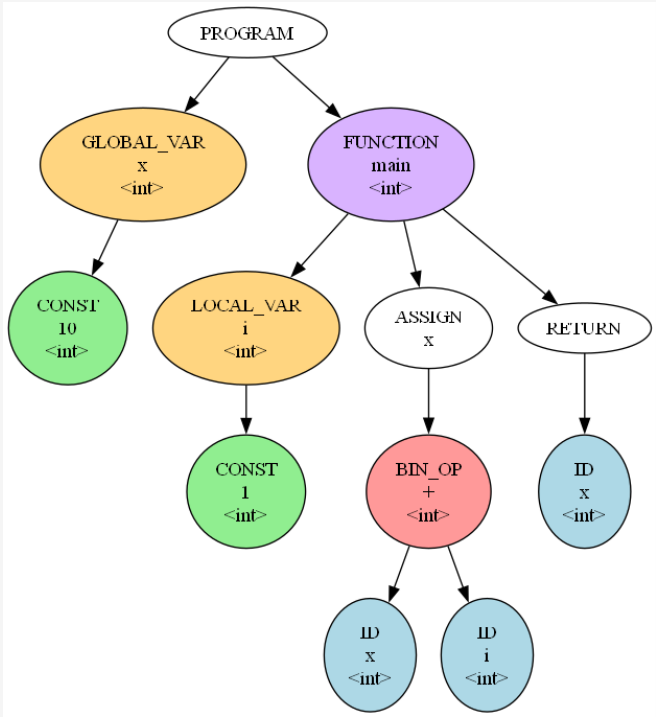
```
int x = 10;

int main() {
    int i = 1;
    x = x + i;
    return x;
}
```

The parser successfully processes the code, ensuring all types are consistent and scopes are respected. The status banner shows `Valid Input`, followed by `Parsing Success!` and `SDT Verified!`.



The corresponding parse tree, automatically generated in the GUI's `output` directory, is shown below:

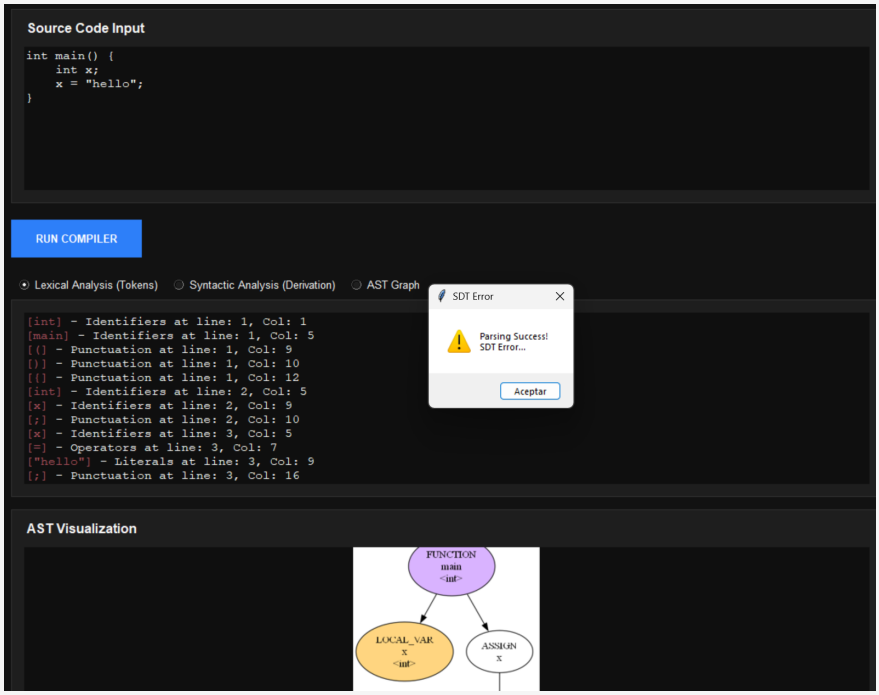


Outcome 2 — Syntactically valid but semantically invalid

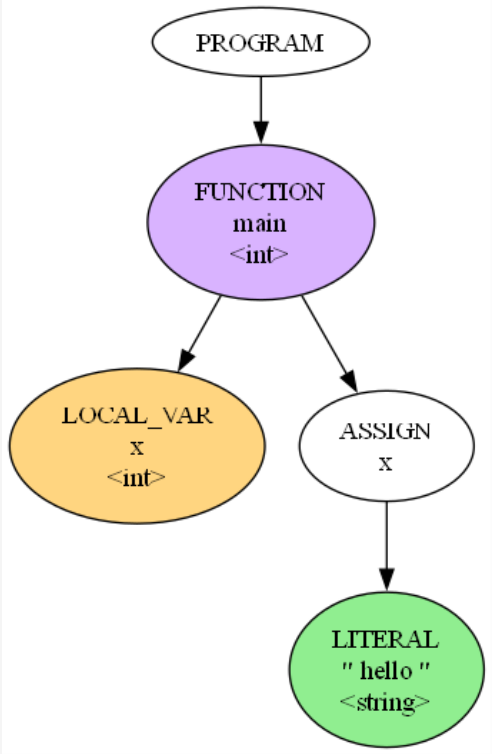
For an input that follows correct grammar structure but violates typing rules (assigning a string literal to an integer variable):

```
int main() {
    int x;
    x = "hello";
}
```

The parser successfully builds the Abstract Syntax Tree, but the Semantic-Directed Translation (SDT) phase detects the type mismatch. The status banner therefore shows `Parsing Success! / SDT error...`.



The corresponding parse tree, automatically generated in the GUI's `output` directory, is shown below:

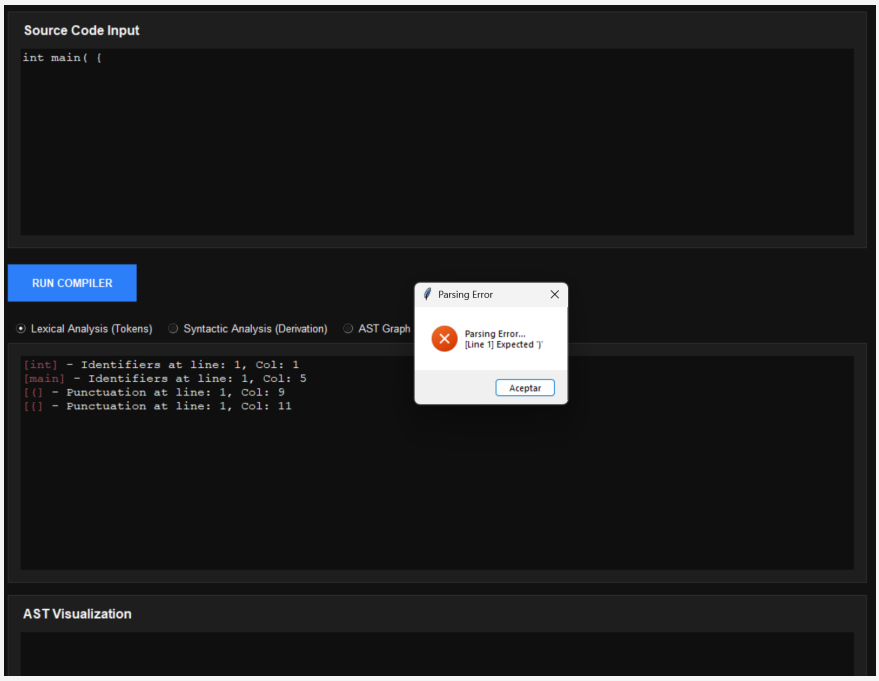


Outcome 3 — Pure syntactic error

For an input containing a structural syntax error (a missing closing parenthesis in the function declaration):

```
int main( {
```

The parser fails immediately during the predictive derivation when it expects a `)` and instead encounters a `{`. The SDT phase is not attempted, and the status banner shows `Parsing error...`.

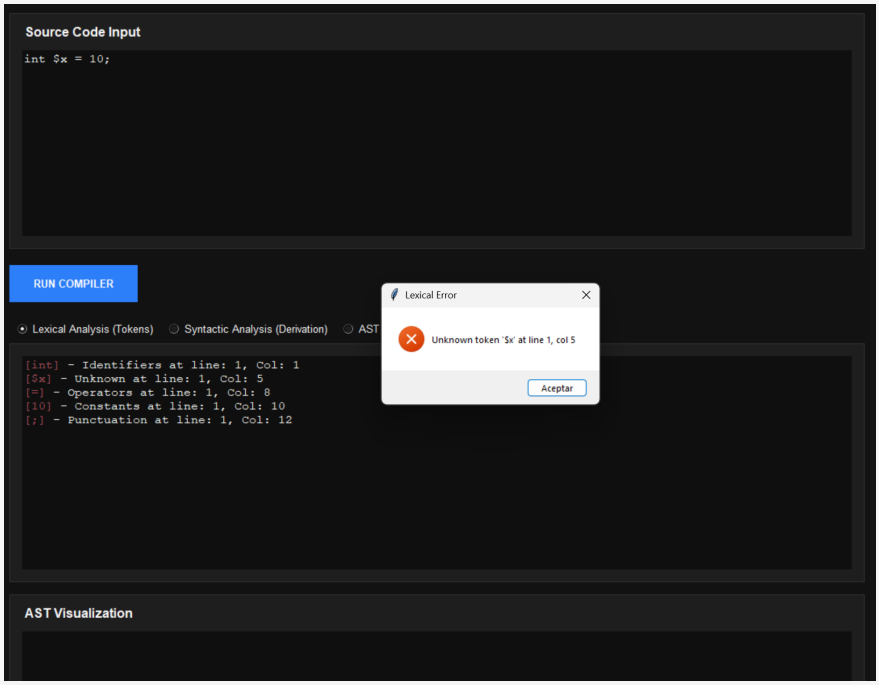


Outcome 4 — Lexical error

For an input containing an illegal or unrecognized character at the initial scanning stage (the `$` symbol within an identifier):

```
int $x = 10;
```

The lexer fails to generate a valid token stream because the character is not defined in the language's alphabet. Execution halts before the parsing stage can even begin, and the status banner shows `Unknown token at line`.



The graphical interface provides a Status panel for the official two-line outcome and three view modes for the lower panel: **Lexical** (token stream with line and column), **Syntactic** (the leftmost derivation produced by the parser) and **Semantic** (the symbol-table stack plus the list of SDT errors, if any).

5. Conclusions

The construction of a parser and an SDT for our C subset confirmed the practical value of the formal techniques studied in class. By applying left factoring (for example, introducing `<STMT_TAIL>` and `<PRIMARY_TAIL>`) and elimination of left recursion (the prime-suffixed non-terminals throughout the expression hierarchy) we obtained an LL(1) grammar whose predictive parser can be coded as a set of mutually recursive procedures, one per non-terminal.

The FIRST and FOLLOW sets, beyond serving as a formal proof of the LL(1) property, also guided the implementation: each `if` branch in the parser corresponds to a FIRST-set test on the current token.

The SDT showed why a parser alone is not enough: a syntactically valid program may still be semantically wrong, and a separate verification step using a stack of scoped symbol tables is needed to catch redeclarations, references to undeclared identifiers, type mismatches in expressions and assignments, and inconsistencies between a function's declared return type and its actual `return` statements.

The clear separation between the lexical, syntactic and semantic stages mirrors the architecture of real compilers and provides a solid base for the later phases of the course (intermediate code generation, optimization and code generation).

References

[1] C. Staff, "What Is Lexical Analysis?," Coursera, Apr. 10, 2024. [Online]. Available: https://www.coursera.org/articles/lexical-analysis

[2] "Introduction of Lexical Analysis," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/compiler-design/introduction-of-lexical-analysis/

[3] "Program to detect tokens in a C code," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/c/c-program-detect-tokens-c-program/

[4] "C Tokens," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/es-es/cpp/c-language/c-tokens?view=msvc-170

[5] "Types of Parsers in Compiler Design," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/compiler-design/types-of-parsers-in-compiler-design/

[6] "Classification of Top Down Parsers," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/compiler-design/classification-of-top-down-parsers/

[7] "FIRST and FOLLOW in Compiler Design," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/compiler-design/why-first-and-follow-in-compiler-design/

[8] "Syntax Directed Translation in Compiler Design," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/compiler-design/syntax-directed-translation-in-compiler-design/